

Real-Time YOLOv8-Based Ripeness Detection and Automated Sorting of Padma Tomatoes with Shelf-Life Estimation

W. K. Gayan, K. R. D. H. Gunawardhana, and P. L. H. Hirushan

Abstract—Manual grading of tomatoes by hand is subjective, varies between operators, and the repeated handling it requires accelerates spoilage. This paper presents an integrated computer-vision and mechatronic system for real-time ripeness grading of the Sri Lankan Padma tomato variety, for which no public detection dataset or conveyor-integrated grading system previously existed. A YOLOv8-medium detector was fine-tuned on a purpose-built dataset of 8,000 images spanning four ripening stages (green, breaker, turning, red) and a defect class, reaching 0.906 precision, 0.922 recall, 0.936 mAP@50 and 0.711 mAP@50–95 on a held-out test split of 787 instances. A confidence-weighted, multi-frame voting policy converts per-frame detections into one classification per tracked fruit, with a deliberately conservative rule for the defect class. Classifications drive an ESP32-controlled conveyor with four presence-sensor-triggered gates, removing dependence on an assumed constant belt speed, and are paired with a per-class remaining shelf-life value. We report an architecture comparison against YOLO26m, a documented dataset-correction cycle in which a lighting confound was exposed only by live-camera testing, live-classification validation, and the system’s current evidence gaps, including an as-yet unmeasured closed-loop sorting trial.

Index Terms—Object detection, YOLOv8, tomato ripeness classification, postharvest handling, conveyor automation, shelf-life estimation, precision agriculture.

I. INTRODUCTION

TOMATO (*Solanum lycopersicum*) is a climacteric fruit: respiration and ethylene production continue after harvest and drive a rapid sequence of chlorophyll loss, carotenoid accumulation and cell-wall softening, after which the fruit bruises easily and becomes vulnerable to decay [1]. In Sri Lanka the Padma variety is an important smallholder cash crop valued for its resistance to bacterial wilt, but fruit reaching collecting centres is still graded by hand: operators sort bulk batches by eye into categories such as green, breaker, turning and red. This manual process is subjective, inconsistent between operators, and requires repeated handling that itself causes skin damage and lets misgraded overripe fruit accelerate the ripening of its neighbours [1].

Convolutional detectors of the YOLO family now classify produce fast enough to run on a moving conveyor [11], [12], but published ripeness models are trained predominantly on other cultivars and imaging conditions and transfer poorly; Nalupano *et al.* [3] demonstrated this directly when a general

architecture had to be retrained specifically for the Philippine Kinalabasa variety. No public dataset exists for Padma fruit annotated across four ripening stages and a defect class, and most published sorting rigs report a model evaluated offline on static images without an integrated, sensor-actuated physical line.

This paper makes three contributions. First, it presents a cultivar-specific detection dataset and a YOLOv8-medium model that localises and classifies Padma fruit into four ripening stages and a defect class. Second, it reports the integration honestly, including a documented case in which the detector learned the imaging conditions of a scarce class rather than the class itself, exposed only once live-camera testing began, together with the correction that fixed it. Third, it links each classification to a measured remaining shelf-life value and line-level operating indicators, and drives physical sorting from presence sensors at each gate rather than from a delay computed from an assumed belt speed.

II. RELATED WORK

Surface colour is the standard basis for tomato grading. The USDA defines six maturity stages by percentage red surface, from green through breaker, turning, pink and light red to red [2]. These bands are defined for reporting, not machine vision, and the two narrowest sit exactly where a camera is least reliable under variable lighting. Generic ripeness standards also travel poorly between cultivars: Nalupano *et al.* [3] retrained MobileNetV2 for the Kinalabasa variety, reaching 94% accuracy but only 64% specificity, illustrating that headline accuracy can conceal weak class separation. For Sri Lanka, Abekoon *et al.* [1] characterised Padma ripening directly, photographing 100 greenhouse-grown fruit daily under simulated market conditions (29°C, 80% relative humidity, 810 lux) to obtain per-stage durations; that work is the source of the shelf-life mapping used here, but it is offline classification of static single-fruit images, with no detector, no localisation and no physical sorting.

Early vision systems used hand-crafted colour descriptors with shallow classifiers [4]–[7]; these are computationally light but degrade outside their tuned imaging conditions. Convolutional networks removed the need to hand-pick descriptors [8]–[10], but classify a whole image rather than localising individual fruit. Single-stage detectors of the YOLO family regress boxes and class probabilities in one forward pass [11], and YOLOv8 in particular is fast enough for real-time operation [12]. Tomato-specific work has adapted this

The authors are with the Department of Industrial Information Technology, Faculty of Applied Sciences, Uva Wellassa University of Sri Lanka, Badulla, Sri Lanka (e-mail: kanushkagayan21@gmail.com).

Manuscript received August 2026.

family mainly to occlusion and lighting variation in field and greenhouse harvesting [13]–[17]; Safira *et al.* [18] compared YOLOv8 directly against a real-time detection transformer for tomato ripeness and provides the most direct basis for preferring a YOLO detector for this task. Almost all of this work targets harvesting robots in cluttered field scenes; a packhouse belt instead presents a controlled background and a single fruit in view, but a decision that must be reached and executed mechanically within seconds.

A detector can also learn whatever separates its classes in training data, including features unrelated to the object itself; this risk is highest for a scarce class topped up from an external source, and the standard mitigations are to hold capture conditions constant and to augment rather than import outside images [19], [20]. This hazard is rarely reported in applied agricultural vision papers, which tend to publish only final metrics.

On actuation, existing sorting rigs use belt sensors chiefly to trigger image capture rather than to release gates, with actuation itself timed from an assumed constant belt speed [21]; such timing error accumulates with distance down the line. Geetha *et al.* [22] link deep-learning grading to shelf-life prediction and a servo-driven sorting prototype, occupying much of the grade-to-shelf-life space, though without cultivar-specific shelf-life measurement or per-gate sensor release. Embedded integration commonly splits inference from actuation across a host and a microcontroller [23], [24], and non-destructive alternatives such as near-infrared spectroscopy [25] and multimodal sensor fusion [26] give internal quality measures that colour imaging cannot, at instrumentation costs a low-cost packhouse rig cannot carry. Shelf life itself is the period over which produce retains acceptable safety and quality under defined storage conditions [27]. What remains absent from the literature is a system in which a cultivar-specific detector, sensor-released (rather than time-predicted) gate actuation, and a measured, cultivar-specific shelf-life mapping are combined on one operating line – the gap this paper addresses.

III. MATERIALS AND METHODS

A. Overall Approach

The system was developed iteratively: problem definition; sample collection; dataset preparation, annotation and splitting; preprocessing and augmentation; model training and testing; dataset correction and retraining where evaluation exposed a problem; hardware design and fabrication; software development; hardware/software integration; and system testing.

B. Materials and Sample Collection

Padma tomatoes were obtained from a commercial collecting centre in Bandarawela, Badulla District, which receives fruit from several farms, giving natural variation in size, texture, maturity and handling damage. A total of 8,000 images were photographed over a 3-day collection period beginning January 2026, with fruit against a white background inside a lightbox under LED illumination, keeping background and

TABLE I
COMPOSITION OF THE DATASET USED FOR MODEL DEVELOPMENT

Property	Value
Classes	5 (breaker, defect, green, red, turning)
Training images	5,498 captured + 668 augmented (defect only)
Validation images	1,574
Test images (held out)	777 (787 instances)
Total distinct captured images	7,849
Split ratio	70.0 / 20.1 / 9.9 %
Input resolution	640×640 px

TABLE II
PER-CLASS ANNOTATED INSTANCE COUNTS BY SPLIT

Class	Train (cap.)	Train (aug.)	Val.	Test	Total
breaker	1,137	0	326	191	1,654
defect	448	896	137	66	1,547
green	1,380	0	429	191	2,000
red	1,434	0	395	190	2,019
turning	1,138	0	309	149	1,596
Total	5,537	896	1,596	787	8,816

lighting constant across sessions and classes – a decision that later proved important (Section III-C1). Images were collected purposively across all five target categories so that the comparatively uncommon defect condition was well represented [28].

The prototype conveyor’s principal components are an ESP32 (DOIT DEVKIT V1) microcontroller; a NEMA 23 stepper drive motor through a 3:1 pulley reduction, commanded by a DM542 step-and-direction driver; four MG995 servo gates with printed diverter blades, one per ripening class; four infrared reflective presence sensors, one at each gate; a 960×720, 30 FPS USB webcam; and a workstation with an NVIDIA RTX 3070 Ti (8,192 MiB VRAM) for training and inference. The software stack comprises Python with PyTorch/Ultralytics (YOLOv8) for training and inference, OpenCV for capture and rendering, Albumentations for offline augmentation, Streamlit/Plotly for the operator dashboard, SQLite for the detection log, Roboflow for annotation and dataset versioning, and PlatformIO/Arduino/ESP32Servo for the firmware.

C. Dataset and Annotation

All images were annotated in Roboflow [29] with a whole-fruit bounding box and a single class label per fruit, exported in YOLO format. The corrected dataset composition is given in Table I and per-class instance counts in Table II; images added to boost the defect class are listed separately from captured images throughout.

The four ripening classes merge adjacent pairs of the six standard USDA maturity stages [2], following Abekoon *et al.* [1]: green (0% red); breaker (<30% red, merging the standard breaker and turning bands); turning (30–80% red, merging pink and light red); and red (>80%). Defect is a separate class covering visible cracking, bruising or rot at any ripening stage,

boxed the same way as the ripening classes so the model also learns a consistent box scale per class.

1) *Annotation Quality Control and Correction:* Annotation went through a review cycle that shaped the final dataset in two steps. First, defect images drawn from a public dataset had been boxed around only the blemish, while project images boxed the whole fruit; every such image was re-annotated to the whole-fruit convention. Second, even after that fix, live-camera testing showed healthy fruit of every ripening class being wrongly classified as defect. The cause was that most defect images still came from the public source, which differed from the other four classes in background and lighting as well as box convention – the model had partly learned the lighting, not the damage, as the sign of a defect. The fix was to drop the mismatched public images, capture and label new defect images under the same lightbox conditions as the other classes, and use offline augmentation to compensate for the smaller resulting pool of real defect photos. Section IV-C reports the effect of both corrections on offline and live performance.

D. Preprocessing, Augmentation and Training

Images are resized to 640×640 px for training and inference. On-the-fly augmentation (colour jitter, rotation, translation, scaling, horizontal flip only, mosaic, mixup, random erasing) follows standard practice for generalising from limited data [19]. The defect class's training images were additionally supplemented offline with Albumentations [20] (flip, rotation, scale, brightness/hue changes), producing two augmented copies per real defect training image, used for training only. A YOLOv8-medium detector, pretrained and fine-tuned end-to-end, was trained locally on the RTX 3070 Ti for up to 150 epochs with early stopping after 25 epochs without validation improvement, an initial learning rate of 0.01, and automatic batch sizing at 640×640 input. A YOLO26-medium model was independently trained on the identical data as a comparison (Section IV-B).

E. Real-Time Detection Pipeline

At run time, live frames (960×720 , 30 FPS) are resized to 640×640 and passed through the detector at a confidence threshold of 0.45, the approximate maximum of the F1-confidence curve. A second, general-purpose detector trained on common everyday object classes filters out tomato detections overlapping non-tomato items. A simple tracker follows one fruit at a time, accumulating a confidence-weighted class vote from each frame (ending a track after 0.6 s with no detection, requiring at least two frames); the final class is the one with the most accumulated weight. The defect class carries one additional, deliberately strict rule – it is only finalised if voted in a clear majority of frames at consistently high confidence – because wrongly rejecting a good fruit loses money permanently, while a borderline fruit that is let through can still be caught later.

F. Hardware Prototype and Control Integration

The conveyor is driven through a 3:1 reduction by the stepper motor described above; four IR sensors and four

servo gates are fitted, one pair per ripening class, encountered downstream from the camera as red, turning, breaker, green. The defect class has no gate and simply proceeds to a single reject stream at the belt end. On classification, the host sends a sequence-numbered gate command over Bluetooth, resent until acknowledged; a controller that has already acted on a sequence number acknowledges again without repeating the action, so that neither a lost command nor a lost acknowledgement misaligns the queue. Each gate maintains its own FIFO queue of pending classifications; when a gate's sensor detects an arriving fruit, the item at the front of that gate's queue is checked and either actuates the gate or is passed to the next gate's queue. Critically, each gate is released by its own sensor rather than by a delay computed from belt speed and camera-to-gate distance, which removes the main source of timing error that would otherwise accumulate for gates further down the belt.

G. Shelf-Life Observation Procedure

Shelf life was measured directly rather than predicted. A total of 100 Padma tomatoes of known starting stage were kept at ambient room temperature over a continuous 31-day period; each morning every fruit was photographed from four angles and placed into one of the four operational stages by colour, with days-in-stage recorded until the fruit became unsellable. The remaining shelf life credited to a detected class is the time left in that stage plus the full length of every later stage – an upper bound, since a single image cannot show how long a fruit has already spent in its current stage – and a defective fruit is assigned zero shelf life. These per-class values form a fixed lookup table applied at the moment of classification and aggregated, with throughput, class distribution and defect ratio, on a live operator dashboard.

H. Experimental and Testing Procedure

Five experiments evaluate the system: (1) held-out test evaluation of detection and classification performance; (2) an architecture comparison between the deployed YOLOv8m and a YOLO26m model trained on identical data; (3) live classification validation inside the lightbox, repeated before and after the correction of Section III-C1; (4) conveyor transport and end-to-end sorting, in a mechanical-only phase and a full closed-loop phase; and (5) the shelf-life observation study of Section III-G. Eight system test cases (Table III) cover detection accuracy, classification, mechanical function and integration.

IV. RESULTS AND DISCUSSION

A. Detection Performance on the Held-Out Test Split

The deployed checkpoint was evaluated once against the 777-image, 787-instance held-out split (Table IV). Training converged in 4.57 hours, with early stopping at epoch 77 restoring the checkpoint from epoch 52; the fused model (93 layers, 25,842,655 parameters, 78.7 GFLOPs) processes a test image in 6.7 ms of inference (0.7 ms pre-, 0.7 ms post-processing) on the RTX 3070 Ti, well inside the 33 ms frame budget at 30 FPS.

TABLE III
SYSTEM TEST CASES

ID	Test description
T-01	Held-out test set detection/classification accuracy
T-02	Architecture comparison on the identical held-out split
T-03	Live classification inside the lightbox
T-04	Conveyor belt mechanical transport
T-05	Non-tomato object rejection
T-06	Single gate actuation bench test
T-07	End-to-end sorting accuracy on the running belt
T-08	Wireless command reliability under retry

TABLE IV
HELD-OUT TEST PERFORMANCE OF THE DEPLOYED YOLOv8M MODEL

Class	P	R	mAP50	mAP50-95
Breaker	0.924	0.937	0.966	0.762
Defect	0.766	0.818	0.804	0.599
Green	0.976	0.990	0.982	0.722
Red	0.987	0.958	0.988	0.724
Turning	0.880	0.906	0.941	0.749
Overall	0.906	0.922	0.936	0.711

Green and red – classes defined by an absolute colour judgement (near-total absence or presence of red surface) – are detected most reliably; breaker and turning, separated from their neighbours by a threshold on a continuous quantity, are detected less reliably, and the defect class least well of all. Figure 1 shows the dominant off-diagonal mass sits between adjacent ripening classes (green↔breaker and breaker↔turning being the largest), not between any ripening class and defect, confirming that residual error is a property of a hard boundary imposed on a continuous ripening gradient rather than a model deficiency. The 66-instance defect row is also the smallest in the test split and should be read with correspondingly wider uncertainty; a bootstrap confidence interval for each cell was not computed within the scope of this study and is left as future work.

B. Architecture Comparison

Table V compares YOLOv8m with a YOLO26m model trained on identical data and evaluated on the identical split; the two runs were not optimised identically (fixed batch size of two for YOLO26m versus automatic batch selection for YOLOv8m, and different early-stopping epochs), so the result establishes which of two concretely trained models suits this task rather than a matched-budget architecture comparison. The two are effectively tied on detection rate (mAP@50 difference of -0.006 , within run-to-run variation for a split this size), while YOLO26m is consistently ahead on mAP@50–95 ($+0.074$ overall), rewarding tighter box localisation. The one exception is instructive: YOLO26m is the only case

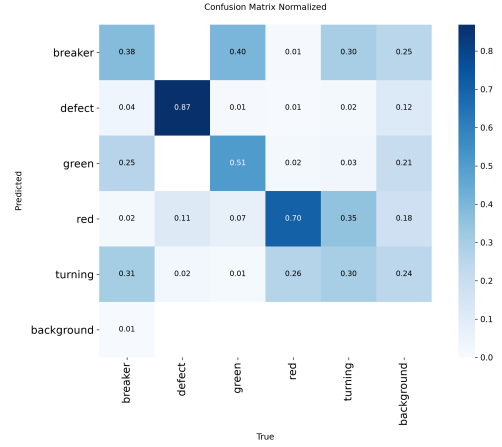


Fig. 1. Row-normalised confusion matrix for the held-out test evaluation.

TABLE V
YOLOv8M VS. YOLO26M ON THE IDENTICAL HELD-OUT SPLIT

Metric	YOLOv8m	YOLO26m	Diff.
Precision	0.906	0.913	+0.007
Recall	0.922	0.911	−0.011
mAP@50	0.936	0.930	−0.006
mAP@50–95	0.711	0.785	+0.074

where the *defect* class loses detection-rate ground (mAP@50 $0.804 \rightarrow 0.754$) while still gaining on mAP@50–95, consistent with the defect class remaining the most fragile part of the dataset – fewest real instances, heaviest reliance on augmentation – rather than any deficiency of either architecture. Since the sorting decision depends on the class label and on the fruit being detected, not on box-edge precision, YOLOv8m was retained for deployment.

C. Effect of the Dataset Corrections

Table VI tracks defect-class and overall mAP@50 across the two corrections of Section III-C1. The first correction alone appears to solve the problem, raising defect mAP@50 from 0.595 to 0.940; this is misleading, because the defect class at that point was still drawn largely from the public source and a same-distribution test split rewards exactly that separability by capture condition. The second correction – recapturing defect fruit under the same lightbox conditions as the other classes – is the one that fixes the underlying problem, and it is also the one that makes the offline number fall, to 0.804: an honest measure of a harder, correctly posed problem, not a regression. The evidence that this correction worked is the live-classification behaviour of Section IV-D, which this offline table alone cannot show.

D. Live Classification Validation

Two live dashboard sessions were logged either side of the imaging-conditions correction (Table VII). Neither followed a formal known-class labelling protocol – both are a log of every classification the dashboard produced against a live feed

TABLE VI
EFFECT OF SUCCESSIVE DATASET CORRECTIONS

Correction stage	Defect mAP50	Overall mAP50
Before correction	0.595	0.903
After box-geometry fix	0.940	0.962
After imaging-conditions fix (deployed)	0.804	0.936

TABLE VII
LIVE DASHBOARD CLASS DISTRIBUTION, BEFORE/AFTER CORRECTION

Class	2026-07-26 (pre)	2026-07-29 (post)
Breaker	1.1%	30.6%
Defect	39.2%	1.7%
Green	17.7%	3.7%
Red	21.0%	58.7%
Turning	21.0%	5.4%
Total classifications	367	242

– so this cannot be reported as a per-class accuracy figure. It nonetheless shows a large, directionally consistent shift: on 2026-07-26 defect accounted for 39.2% of all classifications (mean confidence 0.63), with screenshot evidence showing healthy fruit tagged defect; on 2026-07-29, after the correction, defect fell to 1.7% (mean confidence 0.81), with remaining defect calls corresponding to genuinely damaged fruit. Separately, screenshots from the 2026-07-29 session show three non-tomato objects (a bottle cap, a phone, a wristwatch) classified into a tomato class at 50–81% confidence rather than rejected; whether the non-tomato filter was active in that session was not recorded, so this is reported as an anecdotal observation against test T-05 rather than a controlled result.

E. Conveyor Transport and End-to-End Sorting

The mechanical-transport phase of Experiment 4 confirmed the assembled conveyor transports fruit smoothly and stably at consistent speed without stalling or displacement; fruit count and belt speed were not recorded in this trial. The closed-loop phase – converting live classifications into gate commands and checking physical diversion against known fruit – has not yet been carried out on the assembled rig. This is the single largest remaining evidence gap for the system as a whole, since it is the only planned test that exercises detection, tracking, the command protocol and gate actuation together on physical fruit; the sensor-released gate design of Section III is therefore an engineering justification rather than yet a demonstrated result.

F. Shelf-Life Observation

Table VIII gives the stage durations currently on record for the 100-tomato, 31-day observation. These values match the published Padma stage durations of Abekoon *et al.* [1], mapped through the four-class correspondence, exactly to the day at every boundary. An agreement this precise is more consistent with the published figures having been carried into this project’s records at some point than with two independent

TABLE VIII
RECORDED STAGE DURATION AND DERIVED REMAINING SHELF LIFE

Class	Duration in stage (days)	Remaining shelf life (days)
Green	2	31
Breaker	5	29
Turning	12	24
Red	12	12

100-fruit observational studies coinciding exactly; no raw day-by-day observation record for this project’s own trial was located at the time of writing. We therefore report these values with that caveat attached rather than as a confirmed independent replication, pending recovery of the underlying record.

G. System Test Outcomes

Of the eight planned test cases, T-01 and T-02 passed on the evidence in Sections IVA–B; T-03 and T-04 are partially tested, supported only qualitatively (no ground-truth-labelled accuracy figure and no recorded fruit count/belt speed, respectively); T-05 failed on anecdotal, non-controlled evidence (Section IV-D); and T-06 through T-08 (single-gate bench actuation, end-to-end sorting accuracy, and wireless command reliability under retry) have no recorded trial and are marked not yet tested.

H. Discussion

An overall mAP@50 of 0.936 with precision and recall both above 0.90 shows a detector fine-tuned on a purpose-built Padma dataset localises and classifies this cultivar’s fruit reliably from a held-out split, at a speed the real-time path does not bottleneck on. Published YOLO-based tomato ripeness work reports comparably high figures for other cultivars [13], [18], so this is best read as confirming the architecture family transfers to Padma once trained on Padma-specific data, rather than as a new state of the art on the detection metric alone; the contribution, as scoped in Section II, is the cultivar-specific dataset and the conveyor-integrated system built around it. The ripening-boundary confusion recurs throughout the results – in the single-model test split and as the class most sensitive to the dataset corrections – exactly as a hard multi-class boundary imposed on a continuous colour gradient would predict, suggesting an ordinal or regression-based formulation would match the underlying variable more closely than a flat multi-way classifier. The most transferable finding overall is arguably not a single number but the shape of the correction history in Section IV-C: a scarce class supplemented from an external source can pass every offline check while having learned the wrong signal, and a corrective cycle that stops at re-training and re-scoring on a static split can leave exactly this fault in place – live, in-domain validation was the only check in this study that caught it.

V. CONCLUSION

A YOLOv8m detector fine-tuned on a purpose-built, 8,000-image Padma dataset localises and classifies fruit into four

ripening stages and a defect class with 0.906 precision, 0.922 recall and 0.936 mAP@50 on a held-out split, at inference speeds well within a conveyor line's real-time budget, and is effectively tied with a YOLO26m comparison model on detection rate. Pairing this with sensor-triggered rather than time-predicted gate actuation, a command protocol tolerant of wireless link loss, and a measured shelf-life lookup demonstrates a feasible route from cultivar-specific detection to an operating, logistics-aware sorting line. The corrective cycle documented here – a scarce, externally supplemented class passing offline checks while having learned the wrong signal, caught only by live testing – is a transferable lesson for dataset construction beyond this cultivar.

The main limitations follow directly from the evidence above rather than generic caution. Ripening classes are cut from a continuous variable, so residual confusion between adjacent classes is a property of the class definition, not a deficiency the model could be trained out of. Class boundaries were judged visually rather than measured instrumentally. The defect class rests on only 66 test instances and a training-time augmentation supplement, so its figures carry the widest uncertainty of any class. The shelf-life values reported match a previously published source exactly and require the underlying raw observation record to be confirmed as independent before they can be presented as a fresh measurement. Most significantly, closed-loop physical sorting accuracy on the assembled rig – the test that would exercise detection, tracking, the command protocol and gate actuation together – has not yet been measured. Future work therefore prioritises: completing the closed-loop sorting trial; treating ripeness as an ordinal or regression target rather than a flat multi-way class; grounding class boundaries with colorimetric measurement; splitting the dataset by fruit identity rather than by image to remove near-duplicate leakage; and extending multi-object tracking to lift the current single-file throughput constraint.

ACKNOWLEDGMENT

The authors thank their supervisors at Uva Wellassa University of Sri Lanka for their guidance throughout this project.

REFERENCES

- [1] T. Abekoon, H. Sajindra, J. A. D. C. A. Jayakody, E. R. J. Samarakoon, and U. Rathnayake, "Image processing techniques to identify tomato quality under market conditions," *Smart Agricultural Technology*, vol. 7, p. 100433, 2024.
- [2] United States Department of Agriculture, *United States Standards for Grades of Fresh Tomatoes*. Washington, DC: Fresh Products Branch, Fruit and Vegetable Division, Agricultural Marketing Service, 1991.
- [3] J. M. G. Nalupano, M. P. Omagap, K. J. G. Fortaleza, F. R. B. Ecraela, and J. J. D. Orquia, "Classification of Kinalabasa tomato using convolutional neural network," *Journal of Innovative Technology Convergence*, vol. 6, no. 3, pp. 99–110, 2024.
- [4] K. A. Santoso, A. Kamsyakawuni, and S. V. R. Izza, "Comparison classification of tomatoes ripeness based on RGB, HSV and CMYK colors based on correlation coefficient," *Journal of Computers and Digital Business*, vol. 3, no. 3, pp. 112–120, 2024.
- [5] G. Liu, S. Mao, and J. H. Kim, "A mature-tomato detection algorithm using machine learning and color analysis," *Sensors*, vol. 19, no. 9, p. 2023, 2019.
- [6] M. B. Garcia, S. Ambat, and R. T. Adao, "Tomayto, tomahto: A machine learning approach for tomato ripening stage identification using pixel-based color image classification," in *Proc. IEEE 11th Int. Conf. Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM)*, 2019, pp. 1–6.
- [7] Ifmalinda, Andasuryani, and I. Rasinta, "Identification of tomato ripeness levels (*Lycopersicon esculentum* Mill.) using android-based digital image processing," *IOP Conf. Series: Earth and Environmental Science*, vol. 1182, p. 012003, 2023.
- [8] Y. M. K. Allo, I. P. Paendong, and P. H. Saputro, "Classification of tomato ripeness levels using convolutional neural network (CNN)," *Journal of Intelligent Systems and Information Technology*, vol. 2, no. 2, pp. 80–87, 2025.
- [9] M. C. Centino, V. A. Pitogo, and M. P. B. Pacot, "Tomato maturity assessment: Using convolutional neural networks and image processing-based domain knowledge," *AEIS*, vol. 1, no. 1, pp. 34–45, 2025.
- [10] M. Waseem, M. M. Sajjad, L. H. Naqvi, Y. Majeed, T. U. Rehman, and T. Nadeem, "Deep learning model for precise and rapid prediction of tomato maturity based on image recognition," *Food Physics*, vol. 2, p. 100060, 2025.
- [11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [12] J. Terven and D. Cordova-Esparza, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023.
- [13] Z. Yang, Y. Li, Q. Han, H. Wang, C. Li, and Z. Wu, "A method for tomato ripeness recognition and detection based on an improved YOLOv8 model," *Horticulturae*, vol. 11, no. 1, p. 15, 2025.
- [14] X. Fan and X. Chai, "TRD-Net: An efficient tomato ripeness detection network based on improved YOLO v8 for selective harvesting," *Frontiers in Plant Science*, vol. 17, p. 1748741, 2026.
- [15] W. Huang, Y. Liao, P. Wang, Z. Chen, Z. Yang, L. Xu, and J. Mu, "AITP-YOLO: Improved tomato ripeness detection model based on multiple strategies," *Frontiers in Plant Science*, vol. 16, p. 1596739, 2025.
- [16] J. Ding, Y. Zou, Y. Wang, L. Han, Y. Xiao, R. Zhang, and X. Xi, "A lightweight task-adaptive YOLO for tomato ripeness detection in complex orchard environments," *Horticulturae*, vol. 12, p. 805, 2026.
- [17] D. Wang, Z. Fang, M. Mo, J. Gan, and Z. Sun, "Tomato ripeness detection method based on improved YOLOv11 lightweight model," *Frontiers of Agricultural Science and Engineering*, vol. 13, no. 3, p. 25657, 2026.
- [18] I. Safira, W. Fuadi, and L. Rosnita, "Performance of YOLOv8 algorithm and real-time detection transformer in tomato ripeness detection system," *Journal of Artificial Intelligence and Engineering Applications*, vol. 5, no. 3, 2026.
- [19] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, p. 60, 2019.
- [20] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," *Information*, vol. 11, no. 2, p. 125, 2020.
- [21] V. Moya, M. Guerra, K. Pazmiño, F. Abedrabbo, F. A. Chicaiza, and D. Pozo-Espín, "Tomato classification with YOLOv8: Enhancing automated sorting and quality assessment," *Smart Agricultural Technology*, vol. 12, p. 101221, 2025.
- [22] G. Geetha, P. C. Prabhu Kumar, V. Suriyaraj, and J. Naveen Kumar, "Smart prediction of shelf life and tomato sorting using deep learning," *Asian Journal of Advances in Agricultural Research*, vol. 25, no. 10, pp. 53–67, 2025.
- [23] L. Patria, M. Makhtar, A. Sambas, R. Multajam, A. F. M. Ayob, S. Jamaludin, W. S. M. Sanjaya, and O. Y. Chuan, "Development of a vision-based mobile robot with artificial neural networks (ANN) for classification of tomato ripeness," *Engineering Letters*, vol. 33, no. 6, pp. 32–43, 2025.
- [24] A. Saxena, A. Agarwal, B. Nagrath, C. S. Jayavanth, S. Thulasidoss, S. Maheswari, and P. Sasikumar, "Deep learning-driven IoT solution for smart tomato farming," *Scientific Reports*, vol. 15, p. 31092, 2025.
- [25] K. R. Borba, D. P. Aykas, M. I. Milani, L. A. Colnago, M. D. Ferreira, and L. E. Rodriguez-Saona, "Portable near infrared spectroscopy as a tool for fresh tomato quality control analysis in the field," *Applied Sciences*, vol. 11, no. 7, p. 3209, 2021.
- [26] Y. Liu, C. Wei, S.-C. Yoon, X. Ni, W. Wang, Y. Liu, D. Wang, X. Wang, and X. Guo, "Development of multimodal fusion technology for tomato maturity assessment," *Sensors*, vol. 24, no. 8, p. 2467, 2024.
- [27] F. Tarlak, "The use of predictive microbiology for the prediction of the shelf life of food products," *Foods*, vol. 12, no. 10, p. 2019, 2023.
- [28] I. Etikan, S. A. Musa, and R. S. Alkassim, "Comparison of convenience sampling and purposive sampling," *American Journal of Theoretical and Applied Statistics*, vol. 5, no. 1, pp. 1–4, 2016.
- [29] B. Dwyer, J. Nelson, and T. Hansen, "Roboflow (Version 1.0) [Computer software]," 2024. [Online]. Available: <https://roboflow.com>